

KNN IMPLEMENTATION

Name: Sumit Mund **SIC:** 24BCSH05 **Roll No.:** 15

Topic: K-Nearest Neighbors (KNN) Implementation

KNN FROM SCARTCH

```
In [6]: from collections import Counter
import math
import numpy as np
import pandas as pd

df = pd.read_csv("dataknn1.csv")
print("\n Dataframe Info")
df.info()

train_data = df.iloc[:, 1:-1].values
y = df.iloc[:, -1].values
test_points = [0, 0, 0]

print("\n Split Data Shapes ")
print(f"Train data shape: {train_data.shape}")
print(f"Labels shape: {y.shape}")
print(f"Test point: {test_points}")
```

Dataframe Info
 <class 'pandas.core.frame.DataFrame'>
 RangeIndex: 6 entries, 0 to 5
 Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	obs	6 non-null	int64
1	x1	6 non-null	int64
2	x2	6 non-null	int64
3	x3	6 non-null	int64
4	y	6 non-null	object

 dtypes: int64(4), object(1)
 memory usage: 372.0+ bytes

Split Data Shapes
 Train data shape: (6, 3)
 Labels shape: (6,)
 Test point: [0, 0, 0]

Distance Matrix Calculation (Euclidean Distance)

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

```
In [7]: def euclidean_distance(a, b):
return math.sqrt(sum((a1 - b1) ** 2 for a1, b1 in zip(a, b))) #zip - combine
```

```
In [8]: def knnClassifier(train_data, train_labels, test_point, k):
        dist_list = []

        # Calculate dist from test points
        for i in range(len(train_data)):
            distance = euclidean_distance(test_point, train_data[i])
            dist_list.append((distance, train_labels[i]))
            print(f"Distance {i+1}: {distance}")

        # Sort distances
        dist_list.sort(key=lambda x: x[0])
        print("\nSorted Distance List:")
        print(dist_list)

        # Select k
        k_near = [label for (_, label) in dist_list[:k]]
        print(f"\n{k} Nearest Neighbors:")
        print(k_near)

        # Find majority class
        most_common = Counter(k_near).most_common(1)
        print("\nClass Frequency:")
        print(most_common)

        return most_common[0][0]
```

```
In [9]: # test it
        k = 3
        predicted_class = knnClassifier(train_data, y, test_points, k)
        print(f"\nPredicted Class: {predicted_class}")
```

```
Distance 1: 3.0
Distance 2: 2.0
Distance 3: 3.1622776601683795
Distance 4: 2.23606797749979
Distance 5: 1.4142135623730951
Distance 6: 1.7320508075688772
```

```
Sorted Distance List:
[(1.4142135623730951, 'green'), (1.7320508075688772, 'red'), (2.0, 'red'), (2.23606797749979, 'green'), (3.0, 'red'), (3.1622776601683795, 'red')]
```

```
3 Nearest Neighbors:
['green', 'red', 'red']
```

```
Class Frequency:
[('red', 2)]
```

```
Predicted Class: red
```

KNN WITH SCICKET-LEARN WITH IRIS DATA SET

```
In [13]: #import lib
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris #loading rish dataset from sklearn
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
```

```
In [30]: ## Irish - dataset
iris = load_iris() #in array form
df = pd.DataFrame(iris.data, columns=iris.feature_names) #convert to dataframe
df['target'] = iris.target
print(df)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
..	...	...	...	...	
145	6.7	3.0	5.2	2.3	
146	6.3	2.5	5.0	1.9	
147	6.5	3.0	5.2	2.0	
148	6.2	3.4	5.4	2.3	
149	5.9	3.0	5.1	1.8	

	target
0	0
1	0
2	0
3	0
4	0
..	...
145	2
146	2
147	2
148	2
149	2

[150 rows x 5 columns]

```
In [33]: df.head()
```

```
Out[33]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
In [34]: df.info() #info about dataset
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sepal length (cm)      150 non-null   float64
1   sepal width (cm)       150 non-null   float64
2   petal length (cm)      150 non-null   float64
3   petal width (cm)       150 non-null   float64
4   target                 150 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

Iris Dataset

- **Samples:** 150
- **Features:** 4
- **Classes:** 3
- **Classes:** Setosa, Versicolor, Virginica
- **Features:** Sepal Length, Sepal Width, Petal Length, Petal Width

```
In [38]: # Separate features and Labels
X = iris.data
y = iris.target

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

print(f"Traning data shape {X_train.shape} , testing_data shape {X_test.shape}")
```

Traning data shape (120, 4) , testing_data shape (30, 4)

```
In [39]: # Standardize the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [40]: k = 5
knn = KNeighborsClassifier(n_neighbors=k)

# Train the model
knn.fit(X_train, y_train)

# Predict test data
y_pred = knn.predict(X_test)
```

```
In [41]: # Display predictions
print("\nActual Labels:")
print(y_test)

print("\nPredicted Labels:")
print(y_pred)
```

Actual Labels:

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

Predicted Labels:

[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

In []:

In []: